Assignment 8 : GPU Accelerated Machine Learning

The assignment contains 5 exercises. Each scripts has **DIY Exercise**. Each script is self-contained with:

Section A : Fully provided reference code to study.

Section B : DIY sections with TODO markers and inline hints.

Section C : Stretch goals for advanced students.

Script	Topic	Key Concepts	DIY Tasks
ex01	CUDA Basics	Thread hierarchy, launch config, H2D transfers	Vector scale, squared diff, bandwidth benchmark
ex02	Memory Hierarchy	Shared memory, reduction, bank conflicts	Shared copy, max reduction, histogram
ex03	ML Primitives	Activations, softmax, loss functions	Sigmoid, tanh, ReLU backward, cross - entropy
ex04	CNN Layers	Tiled GEMM, conv2d, batchnorm, pooling	Tiled matmul, max pool, batch norm inference
ex05	MNIST CNN	Full training loop, Adam, LR schedule, AMP	Model architecture, train loop, evaluator

Problem 1: GPU Architecture & CUDA Kernel Profiling

Companion Script	ex01_cuda_basics.cu
Tools Required	CuPy, NVIDIA Nsight Compute (or cudaEvent timing)

Problem Statement

You are tasked with comparing the computational efficiency of a CPU and GPU for element - wise vector operations of increasing size and analyzing the CUDA thread launch configuration.

Part A - Bandwidth & Speedup Analysis

Complete ex01_cuda_basics.cu (all TODO sections). Then run a systematic benchmark:

- Implement the memory bandwidth measurement (TODO B4) for transfer sizes: 1, 8, 64, 256, and 512 MB.
- For vector sizes N = 2^10, 2^14, 2^18, 2^22, 2^26, record: CPU time (ms), GPU compute time (ms), H2D transfer time (ms), achieved GPU speedup.
- Plot two graphs: (a) Time vs N for CPU and GPU, (b) Achieved bandwidth vs transfer size.
- Identify the crossover point where GPU becomes faster than CPU. Explain why small N favors CPU.

Part B - Launch Configuration Analysis

Complete and extend TODO B3 (diy_launch_config):

- For threads_per_block values {64, 128, 256, 512, 1024} and N = 2^20, compute the exact block count and verify all elements are covered.
- Time your vectorAdd kernel with each thread block size. Report on the optimal configuration.
- Explain (in 100 - 150 words) why multiples of 32 are preferred for thread block sizes.

Part C - Stretch: Warp Divergence Experiment

Design a kernel that artificially introduces warp divergence using an if (threadIdx.x % 2 == 0) branch. Measure and compare execution time against a branch - free equivalent.

Expected output

Criterion	Description
B4 bandwidth plot	Correct MB/s values, both H2D and D2H curves present
Speedup table	All 6 N values tested, speedup calculated correctly
Crossover analysis	Correct crossover identified, PCIe overhead explained
Launch config table	All 5 block sizes, correct block count formula
Timing + optimal config	Experiment run, optimal size identified with justification
Warp divergence stretch	Kernel written, timing reported, divergence penalty discussed

Problem 2: Parallel Reduction & Shared Memory Optimization

Companion Script	ex02_memory_hierarchy.cu
Tools Required	CuPy or Numba CUDA

Problem Statement

Implement and profile three reduction strategies naive sequential, shared - memory tree reduction, and warp shuffle reduction and analyze the performance impact of shared memory bank conflicts.

Part A : Three Reduction Strategies

- Complete TODO B2 (max reduction using shared memory tree).
- Complete TODO C1 (warp - level reduction using __shfl_down_sync).
- Implement a naive single - thread sequential reduction as a baseline.
- For N = 2^20, measure execution time for all three strategies. Report time in microseconds and throughput in GB/s.
- Verify correctness of all three approaches against numpy.sum() with atol=0.1.

Part B : Bank Conflict Profiling

Complete TODO B3 (bank conflict demo with strides 1, 2, 4, 8, 16, 32):

- Record kernel execution time for each stride value.
- Explain the bank conflict mechanism for stride=32 and why stride=1 is optimal.
- Propose and implement a padding - based solution (float tile[16][17]) for a 2D shared memory use case. Measure the speedup.

Part C : Histogram with Shared Memory Optimization

Extend TODO B4: implement a faster histogram using per - block private histograms in shared memory (to reduce atomicAdd contention on global memory), then merge the partial results.

Expected output

Criterion	Description
Three reduction kernels	All three implemented, tested, correctness verified
Timing comparison table	Correct times, throughput in GB/s, speedup over naive

Bank conflict analysis	Timing table for all 6 strides, correct explanation
Padding solution	tile[16][17] implemented, speedup measured
Shared - memory histogram	Private histogram per block, merge kernel, correctness check
Report quality	Clear explanations, correct CUDA terminology

Problem 3: Custom ML Kernels - Activations, Loss & Backprop

Companion Script	ex03_ml_primitives.cu
Tools Required	CuPy, PyTorch (for reference comparison)

Problem Statement

Implement a complete set of forward and backward ML primitive kernels and validate them against PyTorch reference implementations. These kernels form the building blocks of a custom deep learning framework.

Part A - Activation Function Suite

- Complete all DIY activation kernels in ex03: sigmoid (B1), tanh (B2), leaky ReLU (B3), ReLU backward (B4).
- For each kernel, verify correctness against numpy/PyTorch with atol ≤ 1e - 4.
- Benchmark all four kernels for N = 10^7 elements. Report: kernel time, memory bandwidth utilization (GB/s), comparison with PyTorch equivalent.
- Plot a single figure showing all activation function curves over the range [- 4, 4] generated by your GPU kernels.

Part B - Loss Functions

- Implement BCE loss (C1) with proper numerical clipping.
- Implement numerically stable cross - entropy (C2) using the log - sum - exp trick.
- Write a test suite: generate random logits and labels, compute loss with your kernel and with PyTorch F.cross_entropy(), and assert that the mean absolute error is below 1e - 4.
- Implement the gradient of cross - entropy (dL/d_logits) as an additional kernel: grad[c] = softmax(logits)[c] - one_hot(label)[c]. Verify against torch.autograd.

Part C - Adam Optimizer Kernel

Implement the Adam update rule as a fused CUDA kernel (all computation in one kernel, no intermediate memory allocations). Verify parameter updates match PyTorch's torch.optim.Adam for 100 steps.

Expected output

Criterion	Description
4 activation kernels correct	All pass ✓ with atol ≤ 1e - 4
Activation benchmarks	Timing and bandwidth reported for all 4
Activation curve plot	All activations on one figure, GPU - generated
BCE + CE kernels correct	Both kernels pass ✓, log - sum - exp used
CE gradient kernel	Correct formula, matches autograd
Adam kernel	Fused kernel, 100 - step match vs PyTorch

Problem 4: Tiled GEMM vs cuBLAS & CNN Layer Benchmarking

Companion Script	ex04_cnn_layers.cu
Tools Required	CuPy, PyTorch, optional cuBLAS via cublasSgemm

Problem Statement

Implement tiled matrix multiplication and benchmark it against the naive and cuBLAS implementations. Then benchmark each CNN layer type (Conv2D, BN, MaxPool, FC) individually for MNIST - sized tensors.

Part A - Tiled GEMM Implementation

- Complete all 5 TODO steps inside tiledMatMul kernel (B1). Verify correctness against numpy.matmul for M=K=N=512.
- Complete the benchmark function (B2): time naive GPU, tiled GPU (TILE=16), and cp.dot (cuBLAS) for matrix sizes: 128, 256, 512, 1024, 2048.
- For each size report: time in ms, achieved GFLOPS (= 2MNK / time_s / 1e9), efficiency vs theoretical peak.
- Draw a roofline plot: x - axis = arithmetic intensity (FLOP/byte), y - axis = GFLOPS. Mark naive and tiled operating points.
- Explain (150 - 200 words) why the tiled kernel underperforms cuBLAS even with shared memory, and what additional optimizations cuBLAS uses (Tensor Cores, FP16, vectorized loads).

Part B - CNN Layer Benchmarks

For input tensors of size [32, 64, 14, 14] (typical MNIST intermediate feature maps):

- Benchmark Conv2D (3x3, same padding): use torch.nn.functional.conv2d vs your naive kernel (if implemented).
- Benchmark BatchNorm (inference mode): your kernel (C2) vs PyTorch BN.
- Benchmark MaxPool(2x2): your kernel (C1) vs PyTorch max_pool2d.
- Report time per layer and build a bar chart: 'Time per CNN layer type.'

Part C - im2col Convolution

Implement im2col as a CUDA kernel: transform input [N,C_in,H,W] into a column matrix [N, C_in*kH*kW, H_out*W_out], then perform GEMM to compute the convolution output. Compare with direct convolution in terms of GFLOPS and memory overhead.

Expected output

Criterion	Description
Tiled GEMM correct	Passes correctness check vs numpy.matmul
GEMM benchmark table	All 3 approaches, 5 sizes, GFLOPS reported
Roofline plot	Correct axis labels, operating points marked
cuBLAS analysis	Tensor Core, vectorized loads mentioned
Layer timing table	3 CNN layers benchmarked, bar chart included
im2col implementation	Kernel correct, memory overhead quantified

Problem 5: Full MNIST CNN Training - Design, Train & Optimize

Companion Script	ex05_mnist_cnn.cu
Tools Required	PyTorch, torchvision, optional: torch.profiler, AMP

Problem Statement

Design, train, and optimize a CNN to classify MNIST handwritten digits achieving at least 97% test accuracy. Systematically ablate design choices (architecture, optimizer, LR schedule, regularization) and document your findings.

Part A - Model Implementation

- Complete all TODO sections in MnistCNN: define layers (B1 - B2) and implement forward() (B3). The model must accept input [N,1,28,28] and output logits [N,10].
- Complete the training loop: data - to - GPU (C1), forward - backward - optimizer step (C2), and evaluate() (C3).
- Complete build_optimizer (D1) and build_scheduler (D2). Try at least two optimizer+scheduler combinations.
- Train for 10 epochs with batch_size=256 and report per - epoch: train loss, train accuracy, test accuracy, GPU memory usage.

Part B - Ablation Study

Train the following 4 configurations for 5 epochs each and record final test accuracy:

- Baseline: ProvidedMnistCNN (given) with Adam, no scheduler, no dropout.
- Add BatchNorm to both conv layers.
- Add Dropout(0.5) before the final FC layer.
- Replace Adam with SGD + Momentum (0.9) + CosineAnnealingLR.

Build a comparison table: Configuration, Test Accuracy (%), Epochs to 95%, Training Time (s).

Discuss in 200 - 250 words which configuration works best and why.

Part C - Data Augmentation

Add random data augmentation to the MNIST training pipeline using torchvision.transforms:

- RandomRotation(±10 degrees)
- RandomAffine (shear up to 10 degrees)
- RandomErasing (probability 0.1)

Train with and without augmentation. Report: final test accuracy, best epoch, and whether augmentation helps.

Part D - Bonus: CUDA Streams & AMP

Implement the stretch sections in ex05:

- Implement async data transfer using torch.cuda.Stream (F). Measure the wall - clock time reduction compared to synchronous transfer for 100 batches.

- Implement FP16 Automatic Mixed Precision using torch.cuda.amp.autocast and GradScaler. Report: training time per epoch, peak GPU memory, final test accuracy.
- Profile the training loop using torch.profiler and identify the top 3 time - consuming operators.

Expected output

Criterion	Description
Model code complete	All TODOs filled, model trains without errors
Training loop correct	Loss decreasing, accuracy ≥ 97% achieved
Ablation table	All 4 configs tested, table and discussion present
Config comparison discussion	BatchNorm/Dropout/optimizer tradeoffs explained
Data augmentation	3 augmentations added, accuracy comparison reported
CUDA Streams (bonus)	Async transfer implemented, speedup measured
AMP training (bonus)	autocast + scaler used, memory/time comparison
Profiler analysis (bonus)	torch.profiler report, top - 3 ops identified

Environment Setup Guide

1. System Requirements

- NVIDIA GPU with CUDA Compute Capability 6.0+ (GTX 1060 or better)
- CUDA Toolkit 12.x - download from developer.nvidia.com/cuda - downloads
- Python 3.10 or newer
- At least 6 GB GPU VRAM for Problem 5

All 5 scripts rewritten as proper CUDA C .cu files compiled with nvcc.

File	Topic	Compile Command
ex01_cuda_basics.cu	**GPU architecture, memory, kernels**	**nvcc -O2 -arch=sm_86 ex01_cuda_basics.cu -o ex01**
ex02_memory_hierarchy.cu	**Shared mem, reduction, bank conflicts**	**same, no extra libs**
ex03_ml_primitives.cu	**Activations, loss, Adam optimizer**	**same + -lm**
ex04_cnn_layers.cu	**Tiled GEMM, MaxPool, BatchNorm**	**same + -lcublas**
ex05_mnist_cnn.cu	**Full cuDNN+cuBLAS MNIST pipeline**	**same + -lcudnn -lcublas**
Makefile	**Build all exercises**	**make all**

Key design of each .cu file:

- Real __global__ kernels, cudaMalloc/cudaFree, cudaMemcpy, CUDA Events for timing
- CUDA_CHECK() macro wraps every API call for proper error handling
- TODO blocks with /* HINT: ... */ comments showing exact code to write
- Exercise 5 uses cuDNN for conv/pool and cuBLAS SGEMM for FC layers true production-grade CUDA.
- For You learning: Check the output implement using python,pycuda, observe the differences